

Repository Structure and Dependency Layering

Status: IMPLEMENTED · enforced by `test/architecture/layering_test.go`

1. Why this is a test and not a convention

A convention about import direction is a convention until something checks it. This one is checked against the real import graph reported by `go list`, so a package that acquires a dependency on a higher layer fails the build rather than being noticed in review eighteen months later — if at all.

Six rules are enforced. Each exists because breaking it causes a specific, recognisable kind of damage.

2. The layers

Dependencies flow **downwards only**. A package may import a lower or equal layer, never a higher one.

#	Layer	What lives there	Damage if it depends upwards
0	foundation	<code>pkg/canonical/jcs</code> , <code>pkg/storage/wal</code> , <code>pkg/storage/snapshot</code> , <code>pkg/platform/telemetry</code>	Foundation code that knows about insurance cannot be reused anywhere else
1	contract	<code>pkg/constitution</code> , <code>pkg/ontology</code> , <code>pkg/contract/event</code> , <code>pkg/platform/audit</code> , <code>pkg/platform/timestamp</code>	A vocabulary that depends on one consumer stops being shared
2	control	<code>pkg/qualification/*</code> , <code>pkg/disclosure/access</code> , <code>pkg/ai/gateway</code> , <code>pkg/authz</code>	A check that depends on what it checks is not a check
3	fabric	<code>pkg/proof</code> , <code>pkg/casefabric</code> , <code>pkg/fref</code>	A fabric that imports a domain has become that domain's
4	domain	<code>pkg/insurance/*</code> , <code>pkg/domain/*</code>	—
5	application	<code>pkg/commercial/*</code> , <code>pkg/api*</code>	—
6	assurance	<code>pkg/assurance</code>	A package that depends on the audit of itself has made the audit part of what it audits

Packages outside this classification are **exempt**, deliberately. The rule can then be tightened package by package rather than demanding the whole repository be reshaped at once — and `TestEveryGovernedLayerHasPackages` fails if a layer ends up governing nothing, so the exemption cannot hollow the rule out.

3. The six rules

Rule	Test	What it prevents
Dependencies flow downwards	<code>TestDependenciesFlowDownwards</code>	The general case
Foundation imports only foundation	<code>TestFoundationImportsOnlyFoundation</code>	A bottom-layer package reaching into an <i>unclassified</i> corner and

Rule	Test	What it prevents
		passing the general rule
The constitution depends on no implementation	TestTheConstitutionDependsOnNoImplementation	An article being quietly weakened to match whatever the code happens to do
Assurance is imported by nothing	TestAssuranceIsImportedByNothing	The audit becoming part of what it audits
No fabric imports a domain	TestNoFabricPackageImportsADomain	Law 1 — one canonical state, many domain projections
Every governed layer has packages	TestEveryGovernedLayerHasPackages	A classification naming packages that do not exist, making the rule pass by testing nothing

On casefabric and the domains

pkg/casefabric/domains.go names pkg/insurance/casestate, pkg/domain/maritime and four others — as **strings**, in its projection table.

That is the point. A string is a reference a test can check (TestEveryCanonicalPackageExists resolves each against the filesystem); an import is a dependency that inverts the relationship and makes the fabric downstream of the domains it is supposed to unify.

4. Directory map

pkg/		
canonical/jcs	RFC 8785 canonicalization and hashing	[foundation]
storage/wal	write-ahead log	[foundation]
platform/telemetry	metrics primitives	[foundation]
constitution	30 executable articles	[contract]
ontology	40 canonical object types, link types	[contract]
contract/event	canonical event envelope, 25 families	[contract]
platform/audit	the one audit ledger	[contract]
platform/timestamp	temporal attestation: chain vs RFC 3161 TSA	[contract]
qualification/	EQF: state, independence, observability, reverseproof, nextbest	[control]
disclosure/access	procedural P0-P5 x content C0-C5, 10 rights	[control]
ai/gateway	the only path by which a model sees evidence	[control]
authz	rights, purpose binding, policy decisions	[control]
proof	the Proof Object and its pipeline	[fabric]
casefabric	the canonical case spine + domain projections	[fabric]
fref	the forward-reverse execution contract	[fabric]
insurance/ domain/	the domains, as projections	[domain]
commercial/	tenant-facing surfaces	[application]
assurance	two-axis status, constitutional proof audit	[assurance]
test/		
architecture	structural rules (this document)	
adversarial	cross-cutting attack cases	
integration	cross-package behaviour	
acceptance e2e soak	stress chaos	
docs/		

adr/	architecture decision records
architecture/	the frozen architecture documents
constitution/	the Evidence Constitution

5. What this does not guarantee

- Layering says nothing about whether a package is *correct*. It says the repository will still be navigable when it is three times this size.
- Exempt packages are genuinely exempt. A dependency between two unclassified packages is unchecked, and there are many.
- The rules are enforced at compile-time granularity, over direct imports. A transitive dependency introduced through an exempt package is invisible to them.